

Homework 3 - Writeup

Team members

Tania Ortiz-Rosales

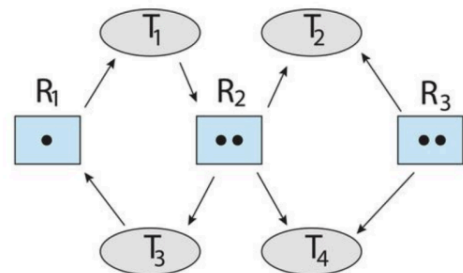
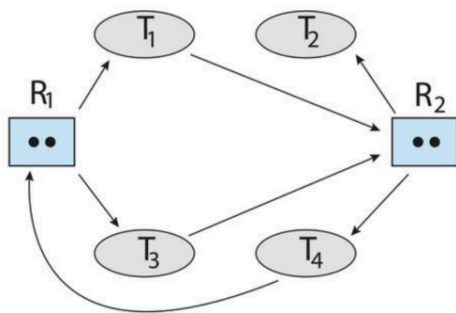
Soniya Pradeepkumar Phaltane

1. [4 pts] Including the initial parent process, how many processes are created by the program shown in the figure below?

```
#include <stdio.h>
#include <unistd.h>
int main() {
    for (int i = 0; i < 3; i++) {
        fork();
    }
    return 0;
}
```

Answer:

- The total amount of processes created by the program above is 8. This is due to the fact that each process is doubled during the fork() call for each iteration. For example, when $i=0$, fork() creates one process so that we have 2 total processes, which is the initial parent process and the first child process. When $i=1$, those 2 processes double and become 4. Finally, when $i=2$, those 4 processes are then doubled again to become a total of 8 processes created by the program.
2. [8 pts] Identify whether a deadlocked situation exists in each graph below, and provide a brief explanation.



Answer:

- (Left Graph): There is no deadlock in this graph. More specifically, there does seem to be a cycle within the RAG graph. However, R2 is able to complete since there is no dependency on T2. Also, there may be an issue between R2 and R1 in terms of timing for the resources but due to T2, it does not indicate that it creates a deadlock scenario.
 - (Right Graph): There is a deadlock in this graph. More specifically, there is a circular wait dependency on R1 -> T1 -> R2 -> T3 -> R1. Due to the circular wait, it implies that there is a cycle within the RAG graph, therefore it implies that the resources are indefinitely waiting for one another so they can't be completed
3. [5 pts] Using the program in the figure below, identify the values of pid at lines A, B, C, and D. (Assume that the actual pids of the parent and child are 2600 and 2603, respectively.)

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid, pid1;
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    } else if (pid == 0) { /* child process */
        pid1 = getpid();
        printf("child: pid = %d",pid); /* A */
        printf("child: pid1 = %d",pid1); /* B */
    } else { /* parent process */
        pid1 = getpid();
        printf("parent: pid = %d",pid); /* C */
        printf("parent: pid1 = %d",pid1); /* D */
        wait(NULL);
    }
    return 0;
}
```

Answer:

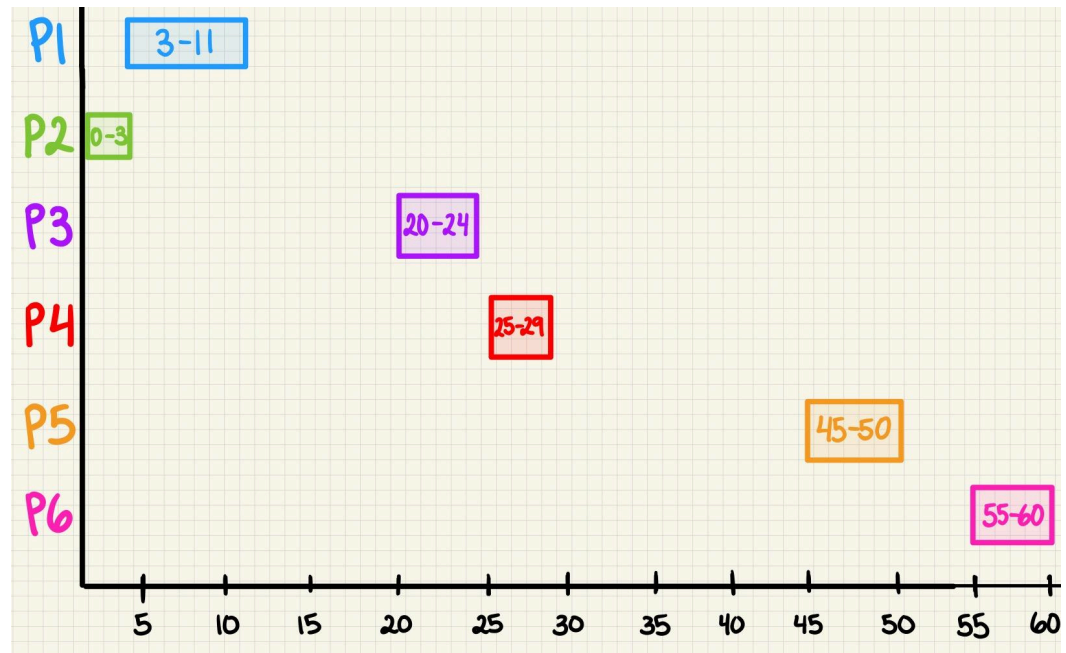
- (A): PID = 0
 - (B): PID = 2603
 - (C): PID = 2603
 - (D): PID = 2600
4. [15 pts] The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm. Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. The scheduler will execute the highest-priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

Process	Burst Time	Priority	Arrival
p1	8	15	0
p2	3	20	0
p3	4	20	20
p4	4	20	25
p5	5	5	45
p6	5	15	55

- a. Show the scheduling order of the processes using a Gantt chart.
- b. What is the turnaround time for each process?
- c. What is the waiting time for each process?

Answer:

- (A)



- (B)

Process	Calculation (completion time - arrival time)	Turnaround time
P1	11-0	11
P2	3-0	3
P3	24-20	4
P4	29-25	4
P5	50-45	5
P6	60-55	5

- (C)

Process	Calculation (turnaround time - burst time)	Waiting time

P1	11-8	3
P2	3-3	0
P3	4-4	0
P4	4-4	0
P5	5-5	0
P6	5-5	0

5. [4 pts] Which of the following scheduling algorithms could result in starvation? a. First-come, first served b. Shortest job first c. Round robin d. Priority

Answer:

- (B) Shortest job first
 - (D) Priority scheduling
6. [4 pts] Explain why interrupt and dispatch latency times must be bounded in a hard real-time system

Answer:

- Interrupt and dispatch latency times must be bounded in a hard real life system because otherwise it could have catastrophic effects in the real-world. For example, if the interrupt latency system is too high in an autonomous car system, then it means that the system does not respond quickly enough and could result in either a missed deadline and failures. If the dispatch latency time is too high, then it could take too long for the scheduler within the system to switch tasks, which will delay the overall execution time. In the previously mentioned AV example, the combination of these two delays could mean that an AV car could potentially not stop for a red light, potentially resulting in another vehicle or pedestrian collision.